

Key Attribute in Entity Framework Core (EF Core)

In this article, I will discuss **Key Data Annotation Attributes in Entity Framework Core (EF Core)** with Examples. Please read our previous article, discussing the [Column Data Annotation Attribute in Entity Framework Core](#) with Examples.

What is the Primary Key in the Database?

A Primary Key is a constraint in a relational database that uniquely identifies each record in a table. It combines the characteristics of both UNIQUE and NOT NULL constraints. This ensures:

- **Uniqueness:** No two rows can have the same values in the primary key columns.
- **Not Null:** Primary key columns cannot contain NULL values.

The primary key plays an important role in enforcing entity integrity by uniquely identifying each record in a table. A table can have only one primary key, which may consist of a single column or multiple columns (referred to as a composite primary key).

Key Attribute in Entity Framework Core

In Entity Framework Core (EF Core), the Key Attribute is a data annotation used to explicitly mark a property as the primary key of a corresponding database table. By default, EF Core tries to infer the primary key based on naming conventions, but the Key Attribute allows us to override this default behavior.

Default Primary Key Convention

By default, EF Core assumes that any property named `Id` or `<EntityName>Id` (case-insensitive) is the primary key. For example, for an entity named `Student`, EF Core will automatically treat properties like `StudentId`, `studentid`, or `ID` as the primary key.

For example, let us modify the **Student.cs** entity as follows. In the below example, EF Core automatically treats `StudentId` as the primary key because of its naming convention. So, this property will be created as a Primary Key column in the corresponding database table.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentId { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
    }
}
```

Modifying DbContext

Next, modify the `DbContext` class, i.e., the `EFCoreDbContext` class, as follows. Here, we are adding the `Student` entity as a `DbSet` Property and providing the property name as `Students` so that the database will create a table with the name `Students`.

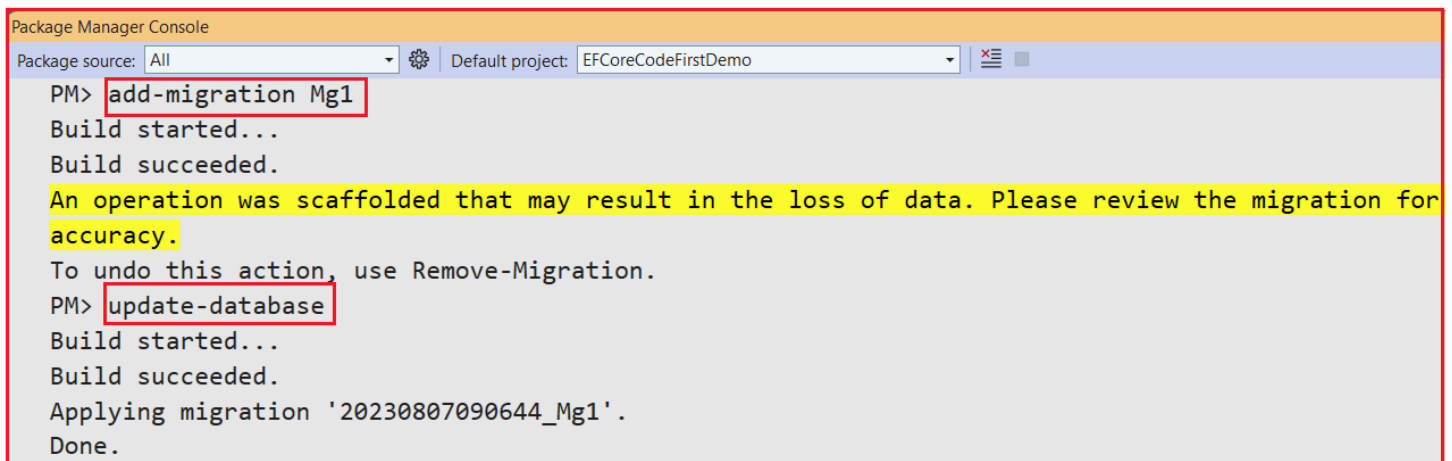
```
using Microsoft.EntityFrameworkCore;
```

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=EFCoreDB;Trusted_Connection=True;TrustServerCertificate=True");
        }
        public DbSet<Student> Students { get; set; }
    }
}

```

As we already discussed, whenever we add or update domain classes or configurations, we need to sync the database with the model using **Add-Migration** and **Update-Database** commands. So, open the Package Manager Console and Execute the **add-migration** and **update-database** commands as follows.

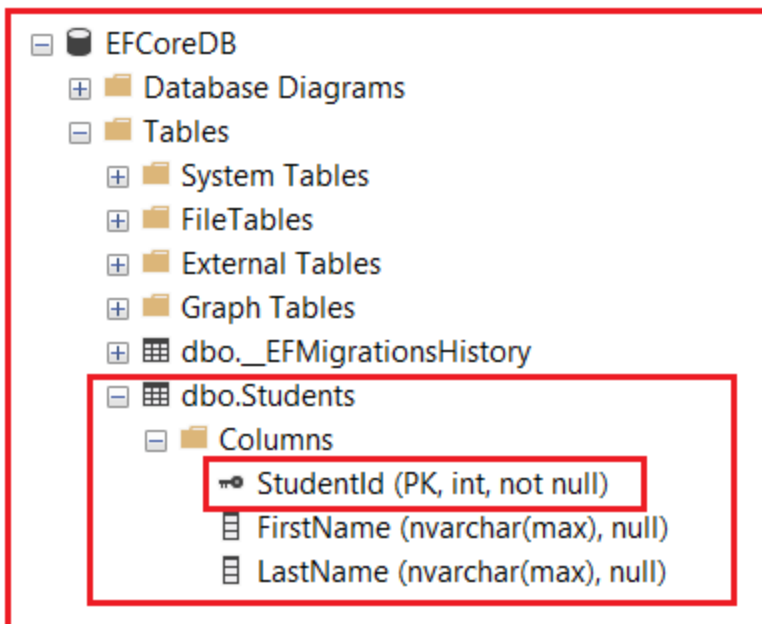


```

Package Manager Console
Package source: All Default project: EFCoreCodeFirstDemo
PM> add-migration Mg1
Build started...
Build succeeded.
An operation was scaffolded that may result in the loss of data. Please review the migration for accuracy.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230807090644_Mg1'.
Done.

```

If you verify the database, you should see the following table. As you can see, the StudentId column is created as the primary key column.

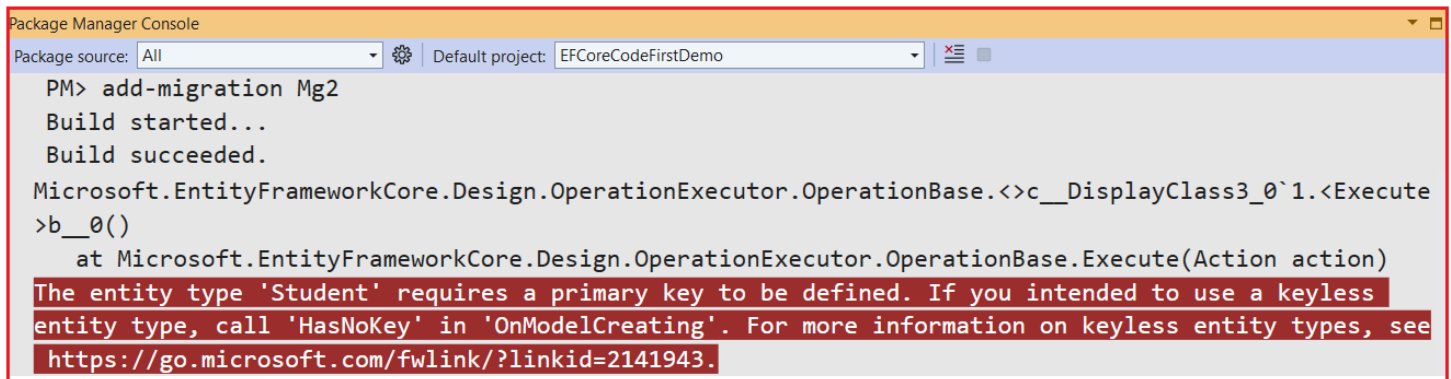


Changing the Primary Key Property

If our business requirement is not to use `StudentId` as the primary key, we can define another property as the primary key using the Key Attribute. We want to create **`StudentRegdNo`** as the primary key column in our database for the Students table. Then, this default convention of EF Core will not work. Let us prove this. Let us modify the Student entity class as follows. Here, instead of `StudentId`, we are using the `StudentRegdNo` property.

```
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public int StudentRegdNo { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
    }
}
```

With the above changes, open the Package Manager Console and Execute the **add-migration** command.



As you can see in the above image, executing the **add-migration** command throws an exception saying the entity type 'Student' requires a primary key to be defined. If you intended to use a keyless entity type, call '**HasNoKey**' in '**OnModelCreating**'.

How to Overcome the Above Error?

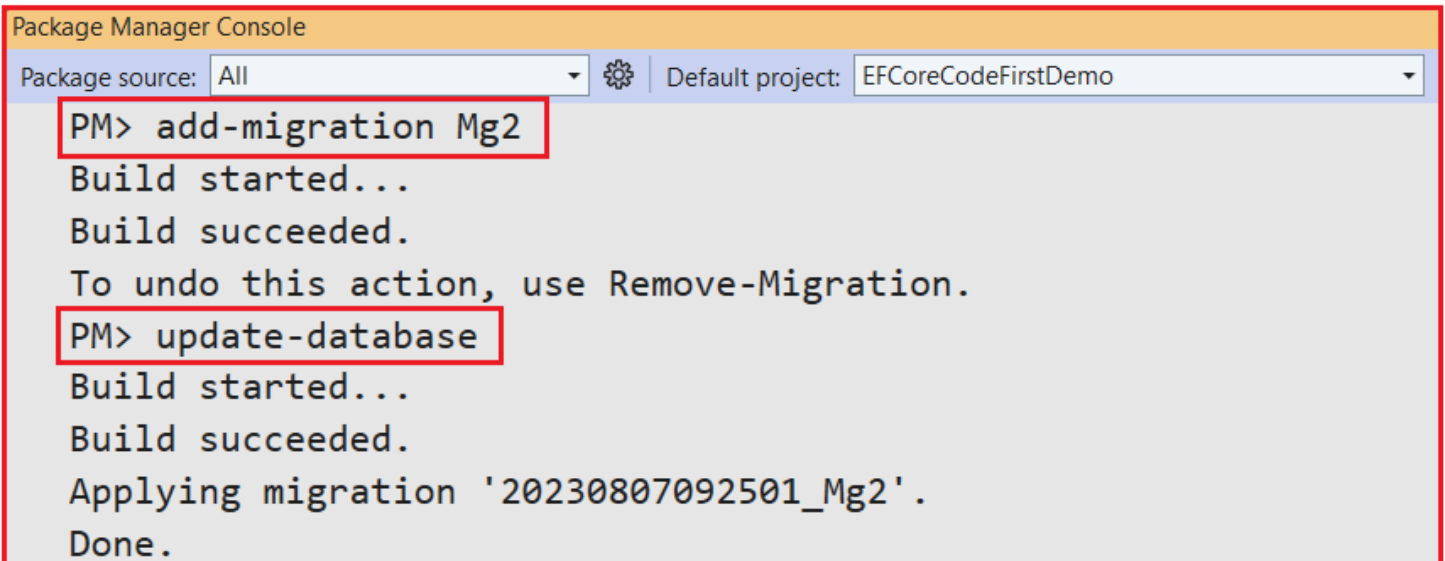
To overcome the above error, we need to mark the **`StudentRegdNo`** property as the key property so that EF Core will make this property the Primary Key column in the corresponding database table. For this, we need to use the **Key Attribute** in EF Core. If you go to the definition of the Key Attribute, you will see the following: It is a sealed class with a parameter-less constructor.

```
namespace System.ComponentModel.DataAnnotations
{
    ... public sealed class KeyAttribute : Attribute
    {
        ... public KeyAttribute();
    }
}
```

So, to override the default behavior and specify a different property as the primary key, we need to use the Key Attribute. For example, if we want **StudentRegdNo** to be the primary, we need to explicitly decorate the **StudentRegdNo** property with **[Key]** as follows. The key Attribute will mark **the StudentRegdNo** property as the key property, and the entity framework core will generate the Primary key for the **StudentRegdNo** property in the database.

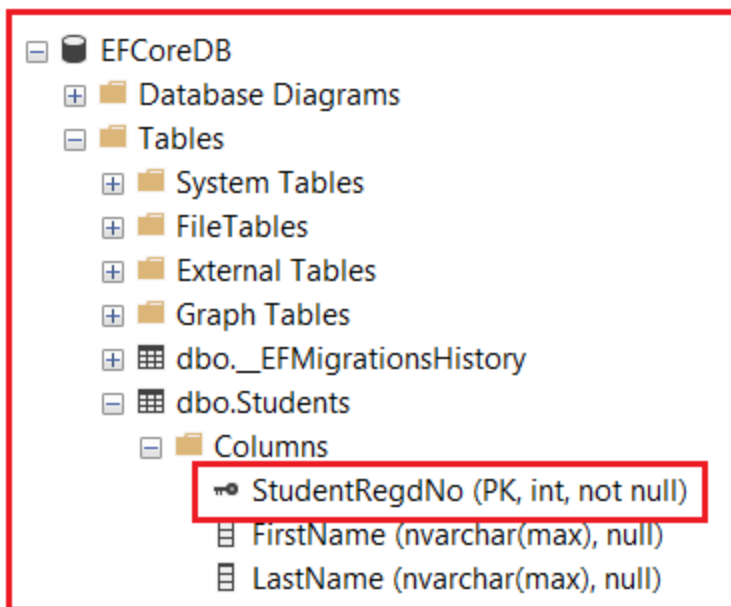
```
using System.ComponentModel.DataAnnotations;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        [Key]
        public int StudentRegdNo { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
    }
}
```

After making the above changes, open the Package Manager Console and Execute the following **Add-Migration** and **Update-Database** commands.



```
Package Manager Console
Package source: All | Default project: EFCoreCodeFirstDemo
PM> add-migration Mg2
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230807092501_Mg2'.
Done.
```

Now, verify the database. You should see **StudentRegdNo** as the primary key, as shown in the image below.



How to Make Composite Primary Key in Entity Framework Core:

In EF Core, we cannot use the `Key` attribute to define a composite primary key. Instead, composite keys must be defined using the **Fluent API** or the **PrimaryKey Attribute** (introduced in EF Core 7.0). If you go to the definition of **PrimaryKey Attribute**, you will see the following: It is a sealed class with one constructor and one property.

```
namespace Microsoft.EntityFrameworkCore;

public sealed class PrimaryKeyAttribute : Attribute
{
    public PrimaryKeyAttribute(string propertyName, params string[] additionalPropertyNames)
    {
        PropertyNames = new List<string> { propertyName };
        foreach (var name in additionalPropertyNames)
        {
            PropertyNames.Add(name);
        }
    }

    public IReadOnlyList<string> PropertyNames { get; }
}
```

PropertyNames:

This property holds the list of property names that constitute the primary key for the entity. The `IReadOnlyList<string>` type ensures that this list is immutable (i.e., it cannot be modified after being set). The list contains the names of the properties in the order in which they are part of the primary key.

Constructor

The constructor takes one required parameter `propertyName` and additional parameters `additionalPropertyNames` (using `params` for flexibility). This allows us to specify a single or composite primary key when applying this attribute.

- **propertyName:** The name of the first property in the primary key.
- **additionalPropertyNames:** Any additional properties that are part of the composite key (if the primary key consists of more than one property).

The constructor initializes the `PropertyNames` list with the provided `propertyName` and adds any additional property names to the list.

Important Notes:

- You cannot apply this PrimaryKey Attribute on an already created database table, and if you try, you will get the error: **System.InvalidOperationException: To change the IDENTITY property of a column, the column needs to be dropped and recreated.**
- The Composite primary keys are configured by placing the [PrimaryKey] attribute on the entity type, not at the property level.

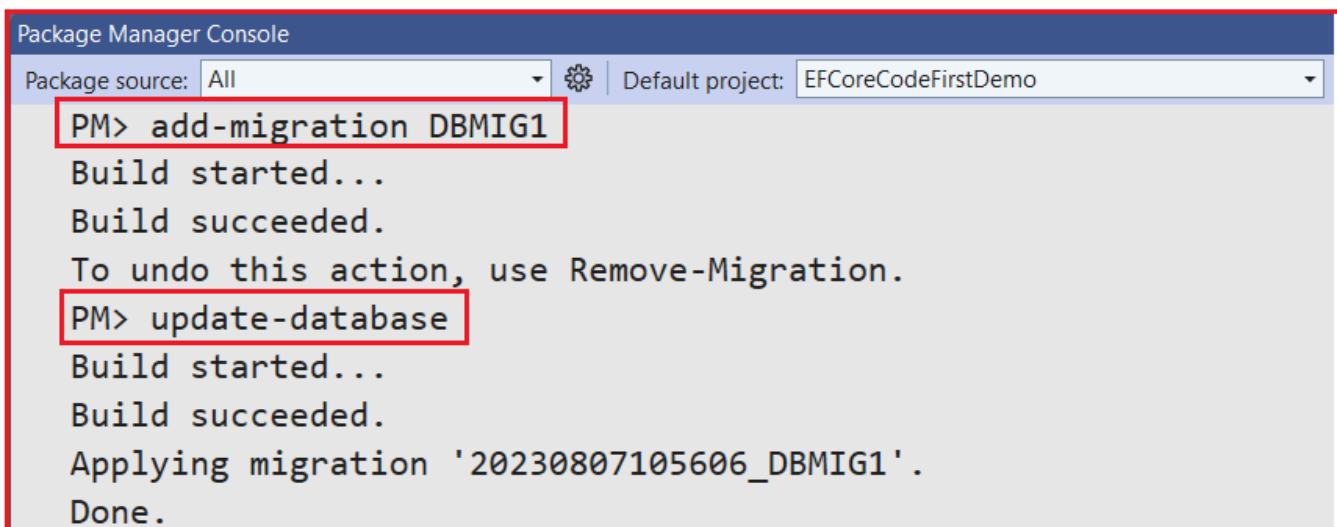
Example to Create Composite Primary Key using EF Core:

So, let us first delete the EFCoreDB database from the SQL Server using SSMS and then delete the Migration folder from our project.

Next, modify the Student Entity Class as follows. As you can see, we have applied the **PrimaryKey** Attribute to the **Student** Entity and passed the **RegdNo** and **SerialNo** properties. This creates a composite key based on RegdNo and SerialNo. EF Core will generate a composite primary key in the database, but neither property will be marked as an identity column.

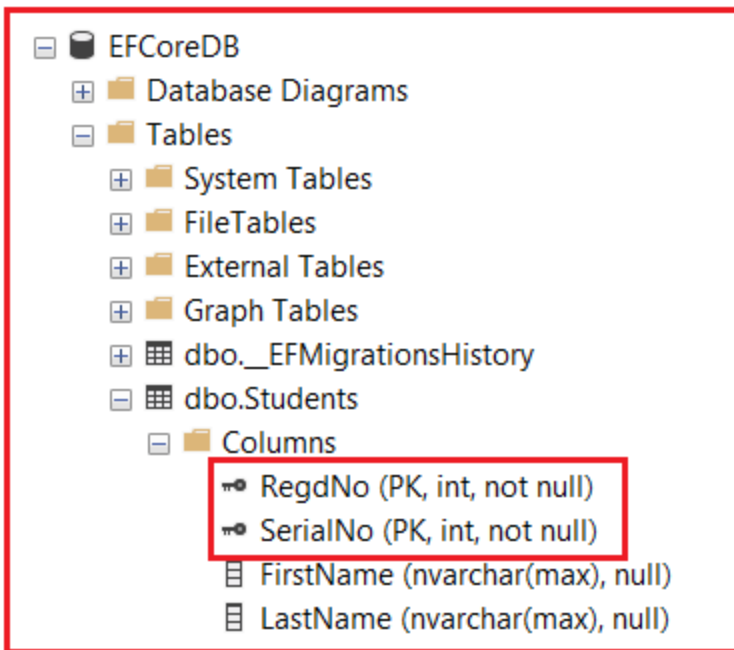
```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [PrimaryKey(nameof(RegdNo), nameof(SerialNo))]
    public class Student
    {
        public int RegdNo { get; set; }
        public int SerialNo { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
    }
}
```

After making the above changes, open the Package Manager Console and Execute the add-migration and update-database commands as follows.



```
Package Manager Console
Package source: All | Default project: EFCoreCodeFirstDemo
PM> add-migration DBMIG1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> update-database
Build started...
Build succeeded.
Applying migration '20230807105606_DBMIG1'.
Done.
```

Now, verify the database. As shown in the image below, you should see a composite primary key based on two columns.



Primary Key on Non-Integer Properties using Entity Framework Core

EF Core allows the creation of a primary key on non-integer properties, including strings. However, when we define a primary key on a string, it will not be created as an identity column. For a better understanding, please modify the Student entity as follows. As you can see, we have created the StudentId property with the string data type, and the EF core will mark this property as the primary key in the database. But this will not be marked as an Identity column.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    public class Student
    {
        public string StudentId { get; set; }
        public string SerialNo { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
    }
}
```

We can also create a composite Primary Key using string properties. For a better understanding, modify the Student entity as follows. As you can see, we have marked the RegdNo and SerialNo properties as String type, and then we create a composite primary key based on these two properties, and it will work.

```
using Microsoft.EntityFrameworkCore;
namespace EFCoreCodeFirstDemo.Entities
{
    [PrimaryKey(nameof(RegdNo), nameof(SerialNo))]
    public class Student
    {
        public string RegdNo { get; set; }
        public string SerialNo { get; set; }
        public string? FirstName { get; set; }
        public string? LastName { get; set; }
    }
}
```

```
}  
}
```

GUID as Primary Key

If you want to use a GUID (Globally Unique Identifier) as a primary key, EF Core also allows this. GUIDs are often used for distributed systems where unique identifiers need to be generated across different servers without collisions. For a better understanding, modify the Student entity as follows:

```
using System.ComponentModel.DataAnnotations;  
namespace EFCoreCodeFirstDemo.Entities  
{  
    public class Student  
    {  
        [Key]  
        public Guid StudentId { get; set; }  
        public string? FirstName { get; set; }  
        public string? LastName { get; set; }  
    }  
}
```

Points to Remember

- **Identity Columns:** Identity columns are created by default for integer-based primary keys. This behavior is disabled for composite keys or non-integer types.
- **Composite Primary Key:** To create a composite primary key, use the Fluent API or the [PrimaryKey] attribute introduced in EF Core 7.0.
- **PrimaryKey Attribute:** The [PrimaryKey] attribute is applied at the entity type level and is useful for defining both single and composite primary keys.
- **Non-Integer Primary Keys:** EF Core allows non-integer properties as primary keys, but they won't be identity columns.

In the next article, I will discuss [Foreign Key Attribute in Entity Framework Core](#) with Examples. I explain the Key Data Annotation Attribute in Entity Framework Core with Examples in this article. I hope you enjoyed this article's Key Data Annotation Attribute in EF Core with Examples. Please provide your valuable feedback and suggestions for this article.